LUIS ROMERO

AI Engineer | Backend Developer (Node.js/TypeScript/Python)

Mérida, Venezuela | +58 424 709 2980 | lueduar15@gmail.com | linkedin.com/in/hernandezrs955 | github.com/ROMEROLUIS15

PROFESSIONAL SUMMARY

AI Engineer and Backend Developer who ships AI products to production and keeps them running there. I build backend for a software company in Texas (United States), where I sustain product delivery with AI-assisted development under my own design and review: 1,050+ pull requests merged in three months. Alongside that I build and operate AI systems with real users every day: WhatsApp, voice and call agents that book appointments autonomously, a workshop assistant on Python/LangGraph serving both customers and back office, and an industrial assistant grounded in real service history. My work is not plugging in an LLM: it is designing the controls that stop it from inventing data or acting on the wrong record, and measuring its reliability with automated evaluations instead of assuming it.

EXPERIENCE

Backend Developer | Complexity (Texas, United States, Remote) July 2026 - Present

Project Factory: backend of a B2B company-directory platform and of its admin panel, with a mobile app. REST API, real time over WebSockets, job queues, full-text search and AI agents (Node.js, NestJS, TypeScript, PostgreSQL, Redis/BullMQ, Elasticsearch, AWS).

- Audited the backend when I joined, presented the findings and got my remediation plan approved, which I led to completion without pausing the roadmap: through incremental refactoring, removed improper coupling between modules, cut circular dependencies from 31 to 0 and moved business logic into 330 use cases on a hexagonal architecture.
- Designed the AI agent harness I work with: numbered rules that the code itself cites, architecture and quality gates, and a check that asks for evidence behind every claim the agent makes (Orca, Gentle AI, MCP, Spec-Driven Development). With it I planned the work in 700+ tickets and delivered 1,050+ merged pull requests and 20+ production releases in three months, reviewing and approving every change.
- Built the quality and security pipeline from scratch. When I joined, no pull request ran any tests. Today every change goes through strict typing, unit, integration and end-to-end tests, security scanning (SAST/SCA, secrets) and architecture rules, and a full PR is validated in ~6 min. Unit tests grew from 20 to 1,000+ suites. The quality check dropped from 9.4 to 3.8 min through parallel execution and integration suites selected from the dependency graph.
- Led the backend of "claim your company" end to end, the product's go-to-market (GTM) program, now in production. It is fed by an AI-assisted loading flow I took to production: an agent builds company profiles and product drafts, and nothing is published without human approval (BullMQ worker, accept-or-reject flow).
- Designed and built an AI extraction engine with current agent-engineering practices: every value carries the quote from its source, two models must agree and code, not the model, decides what gets stored; quality is measured with evals against the human reviewer's decision and traces in Langfuse. It nearly tripled category classification precision.
- Took PostgreSQL row-level security (RLS) to production (63 tables, with pgTAP tests that block the merge), resolved an urgent personal-data exposure report in under 4 hours and operated AWS deployments, closing two production incidents in under an hour.

IBIME Connect (Institutional platform, Remote) January 2025 - July 2026

Software Development Lead (February 2026 - July 2026) | Backend / Full-Stack Developer (January 2025 - February 2026)

- Built the AI assistant a public library network uses to answer citizens' queries, inside the platform that replaced their legacy system. The institution approved it, adopted it as its official platform and, on the back of that delivery, appointed me Software Development Lead.
- Stopped the assistant from making up answers by keeping the model out of the decision flow: a rule-based intent classifier (no LLM) routes the conversation, and if the citizen's email already exists the answer comes from the database without calling the model — less cost, less latency and no opportunity to hallucinate on those queries. A deterministic chain (intent, data protection, match threshold, review and output policy) validates before responding.
- Implemented semantic search over the institution's documents: RAG with PostgreSQL + pgvector and 768-dim Gemini embeddings, inference on Groq gpt-oss-20b configurable per environment, Redis embedding cache, and a match threshold that prefers not answering over answering badly.
- Built a stateful LangGraph agent (extractor, validator and corrector, with bounded retries) that reads the institution's catalog PDFs and turns them into structured, validated data, discarding duplicates by content.
- Hardened the platform for handling citizens' personal data: blocked anonymous writes on sensitive tables (RLS), prevented duplicate registrations, and defined how long each piece of data is kept and how it is deleted. Hexagonal architecture with dependency injection, structured logs and error capture (Sentry); 420+ tests in CI.

Backend Developer | Kiura (Barranquilla, Colombia, Remote) April 2024 - December 2024

- Built the backend services of a teletherapy platform: subscription payments that grant or suspend patient access (MercadoPago via webhook), real-time chat (Socket.IO) and video sessions with per-room signed access (Agora), over a 26-entity domain with dual authentication (JWT + Google OAuth) and role-based permissions.

AI PROJECTS

Founder & AI Engineer | Cronix (in production; operation and maintenance) January 2026 - Present

Multi-tenant SaaS for appointment-based businesses, built and operated single-handedly. In production 24/7: 170+ real appointments handled by the agents and automated reminders, with no manual intervention. 1,600+ tests (Vitest, Playwright, pgTAP) and CI/CD with Sentry.

- Built the WhatsApp and voice agents that book, reschedule and cancel appointments on their own: the customer just asks, at any hour, and nobody at the business has to step in. Custom orchestration engine (no framework), with deterministic routes that resolve most cases without calling the model, and restricted tool access (Groq + Deepgram, 12 capabilities on the voice agent).
- Built a real-time call agent that answers over WhatsApp and books appointments in Cronix by voice (Python, Pipecat Flows, WebRTC, Deepgram): the model talks and code decides every fact and every write, including the caller's confirmation. On Cronix's side, a signed API (HMAC) with idempotency and a lock that prevents booking the same slot twice.
- Chose the call agent's model by measurement, not intuition: on the golden dataset it scored 1.00 on inventing nothing and on no unrequested writes, the first token arrives in 391 ms (p50) and every call is traced in Sentry with its latency and cost.
- Stopped the agent from inventing data or acting on the wrong customer — the real risk of giving an LLM write access to a database full of real clients — with 6 layers of control: keeping the model off the path when the route is deterministic, restricting which tools it may call and with what arguments, anchoring every value to what the user actually said, resolving names against a confidence threshold, forcing a two-step confirmation before any write, and a second model that reviews the operation and can veto it (LLM-as-judge).
- Built the system that checks whether the agent answers well: I saved real conversations as a benchmark and replay them on every change; CI blocks the merge if the agent gets worse (golden conversations), and I also measure it with DeepEval and an LLM judge (relevancy and hallucination).
- Set up the agent's observability: every turn is logged with its outcome, tokens, latency (p50/p95) and errors, visible per business in a custom dashboard on PostgreSQL, with parallel fan-out to LangSmith and error alerting. All of it runs across 9 Edge Functions (Deno).
- Audited my own security until I found a hole in it: I discovered that 13 database functions let one business read another's data, reproduced the attack to confirm it, closed it with an ownership check, and locked it down with 147 automated security tests (pgTAP over 59 isolation policies).

AI Engineer / Backend | Motosmax Cordialidad (Client, Remote) (delivered in phases; in production with support) June 2026 - Present

- Automated both customer service and back-office work for a motorcycle workshop with a Python microservice (FastAPI + LangGraph): one agent serves customers over WhatsApp and another assists the back office with session memory, custom tools, periodic reports and low-stock alerts. The model (Groq/DeepSeek, with fallback) is configurable per environment, and the WhatsApp channel handles Meta's 24h window and surfaces send failures.
- Wired it into the workshop's real system without handing it loose access: the AI microservice talks to the API (NestJS, hexagonal architecture, 37 data models) through credentials that expire in 5 minutes, with sensitive fields encrypted. 400+ tests (Jest, pytest, Playwright) with linting and strict typing in CI; every change starts from a versioned spec (Spec-Driven Development).

AI Engineer / Backend | CMMS Hidrobombas Merida (Client, Remote) (delivered in phases; in production with support) April 2026 - Present

- Built the AI assistant of an industrial maintenance company: it answers by consulting the real report history instead of improvising (LangGraph + RAG persisted on pgvector), with multi-graph orchestration — a tool-using agent plus a diagnostic graph — and the AI layer isolated behind ports so it can be tested without depending on the provider of the day (in-memory vector store in development, pgvector in production). 394 tests, 82% backend coverage.

Advanced Product Search API (The technical challenge that got me into Complexity; deployed and public) July 2026

Live: advanced-search-api-chet.onrender.com/docs · github.com/ROMEROLUIS15/advanced-search-api

- Built the search that ranks results by what the person is actually looking for rather than by insertion order: BM25 relevance on Elasticsearch 8 with typo tolerance, adjusted by popularity and recency, with facet counts computed excluding their own filter (so the user can widen the search instead of hitting zero results), autocomplete and "did you mean" suggestions.
- Delivered it deployed and verified: 500+ tests (97% coverage, including integration against real Elasticsearch), a k6 load test (398,000 requests, 0 failures) and automated security in CI over code, dependencies, git-history secrets, the shipped image and the running API (CodeQL, gitleaks, Trivy + SBOM, OWASP ZAP).

TECHNICAL SKILLS

AI & LLM: LLM Integration, AI Agents, Multi-Agent Orchestration, LangGraph, LangChain, Tool Calling / Function Calling, Structured Outputs, Agentic AI, Vercel AI SDK, OpenAI API, RAG (Retrieval-Augmented Generation), Semantic Search, Embeddings, Vector Databases (pgvector), Prompt Engineering, Anti-Hallucination, AI Guardrails, LLM-as-a-Judge, LLM Evaluation (DeepEval, golden conversations), LLM Observability (LangSmith, Langfuse, traces, tokens, latency), MCP (Model Context Protocol), Voice AI, Pipecat, Pipecat Flows, WebRTC, Speech-to-Text, Text-to-Speech, OpenRouter (STT/TTS), Groq, DeepSeek, Gemini, Deepgram, n8n

Harness Engineering & AI-assisted development: Claude Code (Anthropic), Codex (OpenAI), Orca (ADE), Gentle AI, Engram (persistent agent memory), MCP (Linear, Sentry, AWS, Resend, Figma), multi-agent orchestration, subagents, agent hooks and skills, Context Engineering

Languages & frameworks: TypeScript, JavaScript, Node.js, NestJS, Python, FastAPI, Pydantic, asyncio, uv, Express, Deno, Next.js, React

Backend & data: REST APIs, OpenAPI/Swagger, Transactional email (Resend, React Email), Socket.IO, Microservices, Serverless / Edge Functions, Event-Driven Architecture, Domain Events, Unit of Work, Webhooks, Queues (BullMQ, QStash),

Idempotency, Hexagonal Architecture, Clean Architecture, Architecture Decision Records (ADRs), SOLID, Refactoring, Technical Debt, Dependency Injection, PostgreSQL, SQL, pgvector, Redis, Prisma, TypeORM, Supabase, Neon, Elasticsearch, MySQL, Row-Level Security, Multi-tenant

DevOps, testing & security: Docker, AWS (S3, EC2 Auto Scaling, ECR, CloudWatch, Secrets Manager), Git, GitHub Actions, Husky, Linear, CI/CD (Continuous Integration / Continuous Deployment), Vercel, Render, Cloudflare (R2), Sentry, OpenTelemetry, Observability, Load Testing (k6), pytest, Hypothesis (property-based testing), Snapshot testing, mypy (strict), Ruff, Bandit, pip-audit, Vitest, Jest, Playwright, pgTAP, SAST/DAST/SCA (CodeQL, Semgrep, OWASP ZAP, Trivy, gitleaks, Dependabot), JWT, OAuth 2.0, RBAC, Rate Limiting, HMAC, Threat Modeling, Security Auditing, Spec-Driven Development (SDD)

EDUCATION

Computer Systems Engineering | UPTM Kleber Ramirez - Graduated: Oct 2026

Full-Stack Bootcamp | Academlo - Mar 2024

LANGUAGES

Spanish: Native | English: Intermediate (B1+), advanced technical reading and writing